NAME: Ebenezer

ROLL NO: 2025115168

1. stimulate the **undo and redo option** in a word packages using a **doubly linked list**.

Code:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

struct Node
{
   char text[100];
   struct Node *prev;
   struct Node *next;
};

struct Node *current = NULL;

// Add a new action/text
void typeText(char text[])
{
   struct Node *newNode =
      (struct Node *)malloc(sizeof(struct Node));

   strcpy(newNode->text, text);

   newNode->prev = current;
   newNode->next = NULL;

   // Delete redo history
```

```
    if (current != NULL)
    {
        struct Node *temp = current->next;

        while (temp != NULL)
        {
            struct Node *nextNode = temp->next;
            free(temp);
            temp = nextNode;
        }

        current->next = newNode;
    }

    current = newNode;

    printf("Current Text: %s\n", current->text);
}

// Undo operation
void undo()
{
    if (current == NULL || current->prev == NULL)
    {
        printf("Nothing to Undo\n");
        return;
    }

    current = current->prev;

    printf("After Undo: %s\n", current->text);
```

```
}

// Redo operation
void redo()
{
   if (current == NULL || current->next == NULL)
   {
      printf("Nothing to Redo\n");
      return;
   }

   current = current->next;

   printf("After Redo: %s\n", current->text);
}

int main()
{
   int choice;
   char text[100];

   while (1)
   {
      printf("\n--- WORD PACKAGE ---\n");
      printf("1. Type Text\n");
      printf("2. Undo\n");
      printf("3. Redo\n");
      printf("4. Exit\n");

      printf("Enter choice: ");
      scanf("%d", &choice);
```

```
        getchar();

        switch (choice)
        {
            case 1:
                printf("Enter text: ");
                fgets(text, sizeof(text), stdin);

                text[strcspn(text, "\n")] = '\0';

                typeText(text);
                break;

            case 2:
                undo();
                break;

            case 3:
                redo();
                break;

            case 4:
                return 0;

            default:
                printf("Invalid Choice\n");
        }
    }
}
```

2. Given two sorted lists **L_1 and L_2**, write a procedure to compute **$L_1 \cap L_2$** and **$L_1 \cup L_2$** using basic list operations.

Code:

```
#include <stdio.h>

#include <stdlib.h>


struct Node

{

   int data;

   struct Node *next;

};


// Create a new node

struct Node* createNode(int data)

{

   struct Node *newNode =

      (struct Node*)malloc(sizeof(struct Node));


   newNode->data = data;

   newNode->next = NULL;


   return newNode;

}


// Insert at end

void insertEnd(struct Node **head, int data)

{

   struct Node *newNode = createNode(data);


   if (*head == NULL)

   {
```

```
    *head = newNode;
    return;
  }

  struct Node *temp = *head;

  while (temp->next != NULL)
    temp = temp->next;

  temp->next = newNode;
}

// Find Intersection
struct Node* intersection(struct Node *L1, struct Node *L2)
{
  struct Node *result = NULL;

  while (L1 != NULL && L2 != NULL)
  {
    if (L1->data == L2->data)
    {
      insertEnd(&result, L1->data);

      L1 = L1->next;
      L2 = L2->next;
    }
    else if (L1->data < L2->data)
    {
      L1 = L1->next;
    }
    else
```

```
    {
      L2 = L2->next;
    }
  }

  return result;
}

// Find Union
struct Node* unionList(struct Node *L1, struct Node *L2)
{
  struct Node *result = NULL;

  while (L1 != NULL && L2 != NULL)
  {
    if (L1->data == L2->data)
    {
      insertEnd(&result, L1->data);

      L1 = L1->next;
      L2 = L2->next;
    }
    else if (L1->data < L2->data)
    {
      insertEnd(&result, L1->data);
      L1 = L1->next;
    }
    else
    {
      insertEnd(&result, L2->data);
      L2 = L2->next;
```

```
        }
    }

    // Remaining elements of L1
    while (L1 != NULL)
    {
        insertEnd(&result, L1->data);
        L1 = L1->next;
    }

    // Remaining elements of L2
    while (L2 != NULL)
    {
        insertEnd(&result, L2->data);
        L2 = L2->next;
    }

    return result;
}

// Display list
void display(struct Node *head)
{
    while (head != NULL)
    {
        printf("%d ", head->data);
        head = head->next;
    }

    printf("\n");
}
```

```
int main()
{
    struct Node *L1 = NULL;
    struct Node *L2 = NULL;

    // L1 = 1 2 4 6 8
    insertEnd(&L1, 1);
    insertEnd(&L1, 2);
    insertEnd(&L1, 4);
    insertEnd(&L1, 6);
    insertEnd(&L1, 8);

    // L2 = 2 3 4 7 8
    insertEnd(&L2, 2);
    insertEnd(&L2, 3);
    insertEnd(&L2, 4);
    insertEnd(&L2, 7);
    insertEnd(&L2, 8);

    struct Node *I = intersection(L1, L2);
    struct Node *U = unionList(L1, L2);

    printf("L1: ");
    display(L1);

    printf("L2: ");
    display(L2);

    printf("Intersection: ");
    display(I);
```

```
    printf("Union: ");
    display(U);

    return 0;
}
```

Output:

L1: 1 2 4 6 8

L2: 2 3 4 7 8

Intersection: 2 4 8

Union: 1 2 3 4 6 7 8

3. Swap two elements by adjusting **only the pointers** (and not the data) using a **doubly linked list**.

Code:

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node *prev, *next;
};

struct Node *head = NULL;

// Insert at end
void insert(int data) {
    struct Node *newNode = malloc(sizeof(struct Node));
    newNode->data = data;
    newNode->prev = NULL;
    newNode->next = NULL;
```

```
    if (head == NULL) {
        head = newNode;
        return;
    }

    struct Node *temp = head;
    while (temp->next != NULL)
        temp = temp->next;

    temp->next = newNode;
    newNode->prev = temp;
}

// Swap nodes using pointers
void swap(int x, int y) {
    if (x == y)
        return;

    struct Node *a = head, *b = head;

    while (a != NULL && a->data != x)
        a = a->next;

    while (b != NULL && b->data != y)
        b = b->next;

    if (a == NULL || b == NULL) {
        printf("Node not found\n");
        return;
    }
```

```
// Make a appear before b
struct Node *temp;
for (temp = a; temp != NULL && temp != b; temp = temp->next);

if (temp == NULL) {
   struct Node *t = a;
   a = b;
   b = t;
}

// Adjacent nodes
if (a->next == b) {
   struct Node *p = a->prev;
   struct Node *n = b->next;

   if (p != NULL)
      p->next = b;
   else
      head = b;

   if (n != NULL)
      n->prev = a;

   b->prev = p;
   b->next = a;
   a->prev = b;
   a->next = n;
}
// Non-adjacent nodes
else {
```

```
      struct Node *ap = a->prev;
      struct Node *an = a->next;
      struct Node *bp = b->prev;
      struct Node *bn = b->next;

      if (ap != NULL)
        ap->next = b;
      else
        head = b;

      if (an != NULL)
        an->prev = b;

      if (bp != NULL)
        bp->next = a;
      else
        head = a;

      if (bn != NULL)
        bn->prev = a;

      a->prev = bp;
      a->next = bn;
      b->prev = ap;
      b->next = an;
    }
}

// Display list
void display() {
   struct Node *temp = head;
```

```
    while (temp != NULL) {
        printf("%d ", temp->data);
        temp = temp->next;
    }
    printf("\n");
}

int main() {
    int n, data, x, y;

    printf("Enter number of nodes: ");
    scanf("%d", &n);

    printf("Enter elements: ");
    for (int i = 0; i < n; i++) {
        scanf("%d", &data);
        insert(data);
    }

    printf("Original List: ");
    display();

    printf("Enter two elements to swap: ");
    scanf("%d %d", &x, &y);

    swap(x, y);

    printf("After Swapping: ");
    display();
```

```
    return 0;
}
```

Sample Output

```
Enter number of nodes: 5
Enter elements: 10 20 30 40 50

Original List: 10 20 30 40 50

Enter two elements to swap: 20 40

After Swapping: 10 40 30 20 50
```

4. Write an array implementation of a **self-adjusting list** using a **doubly linked list**, except that **insertions are performed at the end**, and when an element is accessed, it is moved to the **front of the list**, without changing the order of the other elements.

Code:

```
#include <stdio.h>
#define MAX 100

struct Node {
    int data;
    int prev;
    int next;
};

struct Node list[MAX];
int head = -1, tail = -1, count = 0;

// Insert at end
void insert(int value) {
    if (count == MAX) {
        printf("List is full\n");
        return;
```

```
    }

    int i = count++;

    list[i].data = value;
    list[i].prev = tail;
    list[i].next = -1;

    if (head == -1)
        head = i;
    else
        list[tail].next = i;

    tail = i;
}

// Access and move element to front
void accessElement(int value) {
    int i = head;

    while (i != -1 && list[i].data != value)
        i = list[i].next;

    if (i == -1) {
        printf("Element not found\n");
        return;
    }

    if (i == head)
        return;
```

```
    int p = list[i].prev;
    int n = list[i].next;

    // Remove node from current position
    list[p].next = n;

    if (n != -1)
        list[n].prev = p;
    else
        tail = p;

    // Move node to front
    list[i].prev = -1;
    list[i].next = head;
    list[head].prev = i;
    head = i;
}

// Display list
void display() {
    int i = head;

    while (i != -1) {
        printf("%d ", list[i].data);
        i = list[i].next;
    }

    printf("\n");
}

int main() {
```

```
int choice, value;

while (1) {
   printf("\n1. Insert");
   printf("\n2. Access");
   printf("\n3. Display");
   printf("\n4. Exit");

   printf("\nEnter choice: ");
   scanf("%d", &choice);

   switch (choice) {
      case 1:
         printf("Enter value: ");
         scanf("%d", &value);
         insert(value);
         break;

      case 2:
         printf("Enter element to access: ");
         scanf("%d", &value);
         accessElement(value);
         break;

      case 3:
         display();
         break;

      case 4:
         return 0;
```

```
        default:
            printf("Invalid choice\n");
      }
   }
}
```

Sample Output

1. Insert

2. Access

3. Display

4. Exit

Enter choice: 1

Enter value: 10

Enter choice: 1

Enter value: 20

Enter choice: 1

Enter value: 30

Enter choice: 1

Enter value: 40

Enter choice: 3

10 20 30 40

Enter choice: 2

Enter element to access: 30

Enter choice: 3

30 10 20 40

5. Write a c program to **reverse a given doubly linked list**.

Code:

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
   int data;
   struct Node *prev, *next;
};

struct Node *head = NULL;

// Insert at end
void insert(int data) {
   struct Node *newNode = malloc(sizeof(struct Node));

   newNode->data = data;
   newNode->prev = NULL;
   newNode->next = NULL;

   if (head == NULL) {
      head = newNode;
      return;
   }

   struct Node *temp = head;

   while (temp->next != NULL)
      temp = temp->next;

   temp->next = newNode;
```

```
    newNode->prev = temp;
}

// Reverse doubly linked list
void reverse() {
    struct Node *temp = NULL;
    struct Node *current = head;

    while (current != NULL) {
        temp = current->prev;
        current->prev = current->next;
        current->next = temp;

        head = current;
        current = current->prev;
    }
}

// Display list
void display() {
    struct Node *temp = head;

    while (temp != NULL) {
        printf("%d ", temp->data);
        temp = temp->next;
    }

    printf("\n");
}

int main() {
```

```
    int n, data;

    printf("Enter number of nodes: ");
    scanf("%d", &n);

    printf("Enter elements: ");
    for (int i = 0; i < n; i++) {
        scanf("%d", &data);
        insert(data);
    }

    printf("Original List: ");
    display();

    reverse();

    printf("Reversed List: ");
    display();

    return 0;
}
```

Sample Output

```
Enter number of nodes: 5
Enter elements: 10 20 30 40 50

Original List: 10 20 30 40 50
Reversed List: 50 40 30 20 10
```

6. Write a C program to **delete the middle element of a DLL**.

Code:

```
#include <stdio.h>
#include <stdlib.h>
```

```
struct Node {
   int data;
   struct Node *prev, *next;
};

struct Node *head = NULL;

// Insert at end
void insert(int data) {
   struct Node *newNode = malloc(sizeof(struct Node));

   newNode->data = data;
   newNode->prev = NULL;
   newNode->next = NULL;

   if (head == NULL) {
      head = newNode;
      return;
   }

   struct Node *temp = head;

   while (temp->next != NULL)
      temp = temp->next;

   temp->next = newNode;
   newNode->prev = temp;
}

// Delete middle element
```

```
void deleteMiddle() {
    if (head == NULL) {
        printf("List is empty\n");
        return;
    }

    struct Node *slow = head;
    struct Node *fast = head;

    // Find middle node
    while (fast != NULL && fast->next != NULL) {
        slow = slow->next;
        fast = fast->next->next;
    }

    // Update previous node
    if (slow->prev != NULL)
        slow->prev->next = slow->next;
    else
        head = slow->next;

    // Update next node
    if (slow->next != NULL)
        slow->next->prev = slow->prev;

    free(slow);
}

// Display list
void display() {
    struct Node *temp = head;
```

```
    while (temp != NULL) {
        printf("%d ", temp->data);
        temp = temp->next;
    }

    printf("\n");
}

int main() {
    int n, data;

    printf("Enter number of nodes: ");
    scanf("%d", &n);

    printf("Enter elements: ");
    for (int i = 0; i < n; i++) {
        scanf("%d", &data);
        insert(data);
    }

    printf("Original List: ");
    display();

    deleteMiddle();

    printf("After Deleting Middle: ");
    display();

    return 0;
}
```

Sample Output

Enter number of nodes: 5

Enter elements: 10 20 30 40 50

Original List: 10 20 30 40 50

After Deleting Middle: 10 20 40 50

7. Remove duplicate elements from a DLL:

1. **Sorted DLL**
2. **Unsorted DLL**

Code:

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
  int data;
  struct Node *prev, *next;
};

struct Node *head = NULL;

// Insert at end
void insert(int data) {
  struct Node *newNode = malloc(sizeof(struct Node));

  newNode->data = data;
  newNode->prev = NULL;
  newNode->next = NULL;

  if (head == NULL) {
    head = newNode;
    return;
```

```
    }

    struct Node *temp = head;

    while (temp->next != NULL)
        temp = temp->next;

    temp->next = newNode;
    newNode->prev = temp;
}

// Remove duplicates from sorted DLL
void removeDuplicates() {
    struct Node *temp = head;

    while (temp != NULL && temp->next != NULL) {
        if (temp->data == temp->next->data) {
            struct Node *duplicate = temp->next;

            temp->next = duplicate->next;

            if (duplicate->next != NULL)
                duplicate->next->prev = temp;

            free(duplicate);
        }
        else {
            temp = temp->next;
        }
    }
}
```

```
// Display list
void display() {
   struct Node *temp = head;

   while (temp != NULL) {
      printf("%d ", temp->data);
      temp = temp->next;
   }

   printf("\n");
}

int main() {
   int n, data;

   printf("Enter number of nodes: ");
   scanf("%d", &n);

   printf("Enter sorted elements: ");
   for (int i = 0; i < n; i++) {
      scanf("%d", &data);
      insert(data);
   }

   printf("Original List: ");
   display();

   removeDuplicates();

   printf("After Removing Duplicates: ");
```

```
    display();

    return 0;
}
```

Sample Output

Enter number of nodes: 7

Enter sorted elements: 10 10 20 30 30 40 40

Original List: 10 10 20 30 30 40 40

After Removing Duplicates: 10 20 30 40

Code:

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node *prev, *next;
};

struct Node *head = NULL;

// Insert at end
void insert(int data) {
    struct Node *newNode = malloc(sizeof(struct Node));

    newNode->data = data;
    newNode->prev = NULL;
    newNode->next = NULL;
```

```
    if (head == NULL) {
        head = newNode;
        return;
    }

    struct Node *temp = head;

    while (temp->next != NULL)
        temp = temp->next;

    temp->next = newNode;
    newNode->prev = temp;
}

// Remove duplicates from unsorted DLL
void removeDuplicates() {
    struct Node *current = head;

    while (current != NULL) {
        struct Node *temp = current->next;

        while (temp != NULL) {
            if (current->data == temp->data) {
                struct Node *duplicate = temp;

                temp = temp->next;

                duplicate->prev->next = duplicate->next;

                if (duplicate->next != NULL)
                    duplicate->next->prev = duplicate->prev;
```

```
            free(duplicate);
        }
        else {
            temp = temp->next;
        }
      }

      current = current->next;
   }
}

// Display list
void display() {
   struct Node *temp = head;

   while (temp != NULL) {
      printf("%d ", temp->data);
      temp = temp->next;
   }

   printf("\n");
}

int main() {
   int n, data;

   printf("Enter number of nodes: ");
   scanf("%d", &n);

   printf("Enter elements: ");
```

```
    for (int i = 0; i < n; i++) {
        scanf("%d", &data);
        insert(data);
    }

    printf("Original List: ");
    display();

    removeDuplicates();

    printf("After Removing Duplicates: ");
    display();

    return 0;
}
```

Sample Output

Enter number of nodes: 7

Enter elements: 30 10 20 30 40 10 20

Original List: 30 10 20 30 40 10 20

After Removing Duplicates: 30 10 20 40

8. Calculate the **frequency count of a given integer** in a **circular doubly linked list**.

Code:

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node *prev, *next;
};
```

```
struct Node *head = NULL;

// Insert at end
void insert(int data) {
  struct Node *newNode = malloc(sizeof(struct Node));

  newNode->data = data;

  if (head == NULL) {
    head = newNode;
    newNode->next = head;
    newNode->prev = head;
    return;
  }

  struct Node *tail = head->prev;

  tail->next = newNode;
  newNode->prev = tail;
  newNode->next = head;
  head->prev = newNode;
}

// Count frequency
int frequency(int key) {
  if (head == NULL)
    return 0;

  int count = 0;
  struct Node *temp = head;
```

```
    do {
        if (temp->data == key)
            count++;

        temp = temp->next;

    } while (temp != head);

    return count;
}

// Display list
void display() {
    if (head == NULL) {
        printf("List is empty\n");
        return;
    }

    struct Node *temp = head;

    do {
        printf("%d ", temp->data);
        temp = temp->next;

    } while (temp != head);

    printf("\n");
}

int main() {
```

```
    int n, data, key;

    printf("Enter number of nodes: ");
    scanf("%d", &n);

    printf("Enter elements: ");
    for (int i = 0; i < n; i++) {
        scanf("%d", &data);
        insert(data);
    }

    printf("Circular Doubly Linked List: ");
    display();

    printf("Enter element to count: ");
    scanf("%d", &key);

    printf("Frequency of %d = %d\n", key, frequency(key));

    return 0;
}
```

Sample Output

```
Enter number of nodes: 7
Enter elements: 10 20 30 20 40 20 50

Circular Doubly Linked List: 10 20 30 20 40 20 50

Enter element to count: 20

Frequency of 20 = 3
```

9. Search for a **particular element in a binary tree**.

Code:

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
  int data;
  struct Node *left, *right;
};

// Create a new node
struct Node* createNode(int data) {
  struct Node *newNode = malloc(sizeof(struct Node));

  newNode->data = data;
  newNode->left = NULL;
  newNode->right = NULL;

  return newNode;
}

// Search for an element
int search(struct Node *root, int key) {
  if (root == NULL)
    return 0;

  if (root->data == key)
    return 1;

  return search(root->left, key) ||
      search(root->right, key);
```

```
}

int main() {
   int key;

   // Create binary tree
   struct Node *root = createNode(10);

   root->left = createNode(20);
   root->right = createNode(30);

   root->left->left = createNode(40);
   root->left->right = createNode(50);

   root->right->left = createNode(60);
   root->right->right = createNode(70);

   printf("Enter element to search: ");
   scanf("%d", &key);

   if (search(root, key))
      printf("Element found\n");
   else
      printf("Element not found\n");

   return 0;
}
```

Sample Output

```
Enter element to search: 50
Element found
```

10. Calculate the **height or depth of a binary tree** and print it.

Code:

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
  int data;
  struct Node *left, *right;
};

// Create a new node
struct Node* createNode(int data) {
  struct Node *newNode = malloc(sizeof(struct Node));

  newNode->data = data;
  newNode->left = NULL;
  newNode->right = NULL;

  return newNode;
}

// Calculate height of binary tree
int height(struct Node *root) {
  if (root == NULL)
    return 0;

  int leftHeight = height(root->left);
  int rightHeight = height(root->right);

  if (leftHeight > rightHeight)
    return leftHeight + 1;
  else
```

```
    return rightHeight + 1;
}

int main() {
  struct Node *root = createNode(10);

  root->left = createNode(20);
  root->right = createNode(30);

  root->left->left = createNode(40);
  root->left->right = createNode(50);

  root->right->left = createNode(60);
  root->right->right = createNode(70);

  printf("Height of Binary Tree: %d\n", height(root));

  return 0;
}
```

Sample Output

Height of Binary Tree: 3

11. Given a **binary tree**, convert it to a **doubly linked list**, keeping the **same order**.

Code:

```
        #include <stdio.h>
#include <stdlib.h>

struct Node {
  int data;
  struct Node *left, *right;
};
```

```
// Create a new node
struct Node* createNode(int data) {
   struct Node *newNode = malloc(sizeof(struct Node));

   newNode->data = data;
   newNode->left = NULL;
   newNode->right = NULL;

   return newNode;
}

// Convert binary tree to DLL
void convert(struct Node *root,
          struct Node **head,
          struct Node **prev) {

   if (root == NULL)
      return;

   // Process left subtree
   convert(root->left, head, prev);

   // Link current node
   if (*prev == NULL)
      *head = root;
   else {
      (*prev)->right = root;
      root->left = *prev;
   }

   *prev = root;
```

```
    // Process right subtree
    convert(root->right, head, prev);
}

// Display DLL
void display(struct Node *head) {
    while (head != NULL) {
        printf("%d ", head->data);
        head = head->right;
    }
    printf("\n");
}

int main() {
    struct Node *root = createNode(10);

    root->left = createNode(20);
    root->right = createNode(30);

    root->left->left = createNode(40);
    root->left->right = createNode(50);

    root->right->left = createNode(60);
    root->right->right = createNode(70);

    struct Node *head = NULL;
    struct Node *prev = NULL;

    convert(root, &head, &prev);
```

```
    printf("Doubly Linked List: ");
    display(head);

    return 0;
}
```

Sample Output

Doubly Linked List: 40 20 50 10 60 30 70

12. Given a binary tree the task is to connect the nodes that are at the same level, using next right pointers.

Code:

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node *left, *right, *nextRight;
};

// Create a new node
struct Node* createNode(int data) {
    struct Node *newNode = malloc(sizeof(struct Node));

    newNode->data = data;
    newNode->left = NULL;
    newNode->right = NULL;
    newNode->nextRight = NULL;

    return newNode;
}
```

```
// Connect nodes at the same level
void connect(struct Node *root) {
  if (root == NULL)
    return;

  struct Node *queue[100];
  int front = 0, rear = 0;

  queue[rear++] = root;

  while (front < rear) {
    int size = rear - front;
    struct Node *prev = NULL;

    for (int i = 0; i < size; i++) {
      struct Node *current = queue[front++];

      if (prev != NULL)
        prev->nextRight = current;

      prev = current;

      if (current->left != NULL)
        queue[rear++] = current->left;

      if (current->right != NULL)
        queue[rear++] = current->right;
    }

    prev->nextRight = NULL;
  }
```

```
}

// Display nodes using nextRight
void display(struct Node *root) {
  while (root != NULL) {
    struct Node *temp = root;
    struct Node *nextLevel = NULL;

    while (temp != NULL) {
      printf("%d -> ", temp->data);

      if (nextLevel == NULL) {
        if (temp->left != NULL)
          nextLevel = temp->left;
        else if (temp->right != NULL)
          nextLevel = temp->right;
      }

      temp = temp->nextRight;
    }

    printf("NULL\n");
    root = nextLevel;
  }
}

int main() {
  struct Node *root = createNode(10);

  root->left = createNode(20);
  root->right = createNode(30);
```

```
    root->left->left = createNode(40);
    root->left->right = createNode(50);

    root->right->left = createNode(60);
    root->right->right = createNode(70);

    connect(root);

    printf("Nodes connected at same level:\n");
    display(root);

    return 0;
}
```

Sample Output

```
Nodes connected at same level:
10 -> NULL
20 -> 30 -> NULL
40 -> 50 -> 60 -> 70 -> NULL
```

13. Implement priority queue using binary search tree (BST) and perform operation insert

Code:

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node *left, *right;
};

// Create a new node
struct Node* createNode(int data) {
```

```
    struct Node *newNode = malloc(sizeof(struct Node));

    newNode->data = data;
    newNode->left = NULL;
    newNode->right = NULL;

    return newNode;
}

// Insert into priority queue using BST
struct Node* insert(struct Node *root, int data) {
    if (root == NULL)
        return createNode(data);

    if (data < root->data)
        root->left = insert(root->left, data);
    else
        root->right = insert(root->right, data);

    return root;
}

// Display priority queue in sorted order
void inorder(struct Node *root) {
    if (root != NULL) {
        inorder(root->left);
        printf("%d ", root->data);
        inorder(root->right);
    }
}
```

```
int main() {
   struct Node *root = NULL;
   int n, data;

   printf("Enter number of elements: ");
   scanf("%d", &n);

   printf("Enter priorities: ");
   for (int i = 0; i < n; i++) {
      scanf("%d", &data);
      root = insert(root, data);
   }

   printf("Priority Queue: ");
   inorder(root);

   return 0;
}
```

Sample Output

Enter number of elements: 5

Enter priorities: 30 10 50 20 40

Priority Queue: 10 20 30 40 50

14. In a given binary search tree, write a program to delete a node with:

- No children
- One child
- Two children

Code:

```
#include <stdio.h>
#include <stdlib.h>
```

```
struct Node {
  int data;
  struct Node *left, *right;
};

// Create node
struct Node* createNode(int data) {
  struct Node *newNode = malloc(sizeof(struct Node));

  newNode->data = data;
  newNode->left = NULL;
  newNode->right = NULL;

  return newNode;
}

// Insert into BST
struct Node* insert(struct Node *root, int data) {
  if (root == NULL)
    return createNode(data);

  if (data < root->data)
    root->left = insert(root->left, data);
  else if (data > root->data)
    root->right = insert(root->right, data);

  return root;
}

// Find minimum node
struct Node* findMin(struct Node *root) {
```

```
  while (root->left != NULL)
    root = root->left;

  return root;
}

// Delete node from BST
struct Node* deleteNode(struct Node *root, int key) {
  if (root == NULL)
    return NULL;

  if (key < root->data)
    root->left = deleteNode(root->left, key);

  else if (key > root->data)
    root->right = deleteNode(root->right, key);

  else {
    // Case 1: No children
    if (root->left == NULL && root->right == NULL) {
      free(root);
      return NULL;
    }

    // Case 2: One child
    else if (root->left == NULL) {
      struct Node *temp = root->right;
      free(root);
      return temp;
    }
```

```
    else if (root->right == NULL) {
      struct Node *temp = root->left;
      free(root);
      return temp;
    }

    // Case 3: Two children
    else {
      struct Node *temp = findMin(root->right);

      root->data = temp->data;

      root->right = deleteNode(root->right, temp->data);
    }
  }

  return root;
}

// Inorder traversal
void inorder(struct Node *root) {
  if (root != NULL) {
    inorder(root->left);
    printf("%d ", root->data);
    inorder(root->right);
  }
}

int main() {
  struct Node *root = NULL;
  int n, data, key;
```

```
    printf("Enter number of nodes: ");
    scanf("%d", &n);

    printf("Enter elements: ");
    for (int i = 0; i < n; i++) {
        scanf("%d", &data);
        root = insert(root, data);
    }

    printf("Original BST: ");
    inorder(root);

    printf("\nEnter element to delete: ");
    scanf("%d", &key);

    root = deleteNode(root, key);

    printf("BST after deletion: ");
    inorder(root);

    return 0;
}
```

Sample Output

```
Enter number of nodes: 7
Enter elements: 50 30 70 20 40 60 80

Original BST: 20 30 40 50 60 70 80

Enter element to delete: 50
```

BST after deletion: 20 30 40 60 70 80

15. Convert the given postfix expression into an expression tree.

Code:

```
#include <stdio.h>
#include <stdlib.h>
#include <ctype.h>
#include <string.h>

struct Node {
   char data;
   struct Node *left, *right;
};

struct Node *stack[100];
int top = -1;

// Create node
struct Node* createNode(char data) {
   struct Node *newNode = malloc(sizeof(struct Node));

   newNode->data = data;
   newNode->left = NULL;
   newNode->right = NULL;

   return newNode;
}

// Push into stack
void push(struct Node *node) {
   stack[++top] = node;
```

```
}

// Pop from stack
struct Node* pop() {
  return stack[top--];
}

// Construct expression tree
struct Node* constructTree(char postfix[]) {
  struct Node *node;

  for (int i = 0; postfix[i] != '\0'; i++) {
    char ch = postfix[i];

    if (isalnum((unsigned char)ch)) {
      node = createNode(ch);
      push(node);
    }
    else if (ch == '+' || ch == '-' ||
          ch == '*' || ch == '/' || ch == '^') {

      node = createNode(ch);

      node->right = pop();
      node->left = pop();

      push(node);
    }
  }

  return pop();
```

```
}

// Inorder traversal
void inorder(struct Node *root) {
   if (root != NULL) {
      if (root->left != NULL || root->right != NULL)
         printf("(");

      inorder(root->left);
      printf("%c", root->data);
      inorder(root->right);

      if (root->left != NULL || root->right != NULL)
         printf(")");
   }
}

// Preorder traversal
void preorder(struct Node *root) {
   if (root != NULL) {
      printf("%c", root->data);
      preorder(root->left);
      preorder(root->right);
   }
}

// Postorder traversal
void postorder(struct Node *root) {
   if (root != NULL) {
      postorder(root->left);
      postorder(root->right);
```

```
        printf("%c", root->data);
    }
}

int main() {
    char postfix[100];

    printf("Enter postfix expression: ");
    scanf("%99s", postfix);

    struct Node *root = constructTree(postfix);

    printf("Infix Expression: ");
    inorder(root);

    printf("\nPrefix Expression: ");
    preorder(root);

    printf("\nPostfix Expression: ");
    postorder(root);

    return 0;
}
```

Sample Output

```
Enter postfix expression: ab+c*

Infix Expression: ((a+b)*c)
Prefix Expression: *+abc
Postfix Expression: ab+c*
```

16. Construct a binary search tree using the following traversals:
    1. Inorder traversal

2. Preorder traversal
3. Postorder traversal

Code:

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
  int data;
  struct Node *left, *right;
};

// Create a new node
struct Node* createNode(int data) {
  struct Node *newNode = malloc(sizeof(struct Node));

  newNode->data = data;
  newNode->left = NULL;
  newNode->right = NULL;

  return newNode;
}

// Insert into BST
struct Node* insert(struct Node *root, int data) {
  if (root == NULL)
    return createNode(data);

  if (data < root->data)
    root->left = insert(root->left, data);
  else if (data > root->data)
    root->right = insert(root->right, data);
```

```
    return root;
}

// Inorder traversal
void inorder(struct Node *root) {
    if (root != NULL) {
        inorder(root->left);
        printf("%d ", root->data);
        inorder(root->right);
    }
}

// Preorder traversal
void preorder(struct Node *root) {
    if (root != NULL) {
        printf("%d ", root->data);
        preorder(root->left);
        preorder(root->right);
    }
}

// Postorder traversal
void postorder(struct Node *root) {
    if (root != NULL) {
        postorder(root->left);
        postorder(root->right);
        printf("%d ", root->data);
    }
}
```

```
int main() {
   struct Node *root = NULL;
   int n, data;

   printf("Enter number of nodes: ");
   scanf("%d", &n);

   printf("Enter elements: ");
   for (int i = 0; i < n; i++) {
      scanf("%d", &data);
      root = insert(root, data);
   }

   printf("\nInorder Traversal: ");
   inorder(root);

   printf("\nPreorder Traversal: ");
   preorder(root);

   printf("\nPostorder Traversal: ");
   postorder(root);

   return 0;
}
```

Sample Output

```
Enter number of nodes: 7
Enter elements: 50 30 70 20 40 60 80

Inorder Traversal: 20 30 40 50 60 70 80
Preorder Traversal: 50 30 20 40 70 60 80
Postorder Traversal: 20 40 30 60 80 70 50
```

17. Implement Heap Sort for a given set of numbers.

The numbers shown in the image appear to be:

- Input: 50, 20, 30, 25, 15, 12, 22,32
- Heap: 50, 25, 32, 20, 15, 22, 30

Code:

```
#include <stdio.h>

// Heapify function
void heapify(int a[], int n, int i) {
   int largest = i;
   int left = 2 * i + 1;
   int right = 2 * i + 2;

   if (left < n && a[left] > a[largest])
      largest = left;

   if (right < n && a[right] > a[largest])
      largest = right;

   if (largest != i) {
      int temp = a[i];
      a[i] = a[largest];
      a[largest] = temp;

      heapify(a, n, largest);
   }
}

// Heap Sort
void heapSort(int a[], int n) {
```

```
    // Build Max Heap
    for (int i = n / 2 - 1; i >= 0; i--)
        heapify(a, n, i);

    // Extract elements
    for (int i = n - 1; i > 0; i--) {
        int temp = a[0];
        a[0] = a[i];
        a[i] = temp;

        heapify(a, i, 0);
    }
}

// Display array
void display(int a[], int n) {
    for (int i = 0; i < n; i++)
        printf("%d ", a[i]);

    printf("\n");
}

int main() {
    int n, a[100];

    printf("Enter number of elements: ");
    scanf("%d", &n);

    if (n < 1 || n > 100) {
        printf("Invalid number of elements\n");
        return 1;
```

```
    }

    printf("Enter elements: ");
    for (int i = 0; i < n; i++)
        scanf("%d", &a[i]);

    printf("Original Array: ");
    display(a, n);

    heapSort(a, n);

    printf("Sorted Array: ");
    display(a, n);

    return 0;
}
```

Sample Output

```
Enter number of elements: 8
Enter elements: 50 20 30 25 15 12 22 32

Original Array: 50 20 30 25 15 12 22 32
Sorted Array: 12 15 20 22 25 30 32 50
```